

-- BASIC TO INTERMEDIATE

1. List all customers and the total number of orders they have placed.
Show only customers with more than 5 orders. Sort by total orders descending.

```
-- retrieve customer specific group by customer data
-- filter order cnt using having
select c.customerid,c.companyname,c.contactname, count(o.orderid) as
total_orders
from customers c join orders o on c.customerid = o.customerid
group by c.customerid,c.companyname,c.contactname
having count(o.orderid) > 5
order by total_orders desc;
```

	customerid [PK] character (5)	companyname character varying (40)	contactname character varying (30)	total_orders bigint
1	SAVEA	Save-a-lot Markets	Jose Pavarotti	31
2	ERNSH	Ernst Handel	Roland Mendel	30
3	QUICK	QUICK-Stop	Horst Kloss	28
4	FOLKO	Folk och få HB	Maria Larsson	19
5	HUNGO	Hungry Owl All-Night Grocers	Patricia McKenna	19

```
-- 2. Retrieve the total sales amount per customer by joining customers, orders,
-- and order_details.
-- Show only customers whose total sales exceed 10,000. Sort by total sales
-- descending.
```

```
-- select customer data attributes join customers->orders->orderdetails
-- calculate total sales and the filter sales by having
select
c.customerid,c.companyname,c.contactname,sum(d.unitprice*d.quantity*(1-d.discount)) as total_sales
from customers c join orders o on c.customerid = o.customerid
join order_details d on o.orderid = d.orderid
group by c.customerid,c.companyname,c.contactname
having sum(d.unitprice*d.quantity*(1-d.discount)) > 10000
order by total_sales desc;
```

Data Output Messages Notifications				
	customerid [PK] character (5)	companyname character varying (40)	contactname character varying (30)	total_sales double precision
1	QUICK	QUICK-Stop	Horst Kloss	110277.30488350891
2	ERNSH	Ernst Handel	Roland Mendel	104874.97835350106
3	SAVEA	Save-a-lot Markets	Jose Pavarotti	104361.94986812412
4	RATTC	Rattlesnake Canyon Grocery	Paula Wilson	51097.80048773782
5	HUNGO	Hungry Owl All-Night Groc...	Patricia McKenna	49979.90494634077

-- 3. Get the number of products per category.
 -- Show only categories having more than 10 products. Sort by product count
 -- descending.

-- select category attributes join categories->products
 -- group by and filter using having

```
select c.categoryid,c.categoryname,count(p.productid) as total_products
from categories c join products p on c.categoryid = p.categoryid
group by c.categoryid,c.categoryname
having count(productid)>10;
```

	categoryid [PK] integer	categoryname character varying (15)	total_products bigint
1	2	Condiments	12
2	3	Confections	13
3	1	Beverages	12
4	8	Seafood	12

- 4. Display the total quantity sold per product.
- Include only products where total quantity sold is greater than 100. Sort by
- quantity descending.

```
-- select appropriate product attributes and then calculate sum of quantity
-- group by products and filter using having
select p.productid,p.productname,sum(d.quantity) as total_quantity
from products p join order_details d on p.productid = d.productid
group by p.productid,p.productname
having sum(d.quantity) > 100
order by total_quantity desc;
```

	productid [PK] integer	productname character varying (40)	total_quantity bigint
1	60	Camembert Pierrot	1577
2	59	Raclette Courdavault	1496
3	31	Gorgonzola Telino	1397
4	56	Gnocchi di nonna Alice	1263
5	16	Pavlova	1158

- 5. Find the total number of orders handled by each employee.
- Show only employees who handled more than 20 orders. Sort by order count
- descending.

```
-- select employee attributes ,group by them and then filter using having
-- order by the total orders cnt
select e.employeeid,e.firstname,e.lastname,count(o.orderid) as total_orders
from employees e join orders o on e.employeeid = o.employeeid
group by e.employeeid,e.firstname,e.lastname
having count(o.orderid) > 20
order by total_orders desc;
```

	employeeid [PK] integer	firstname character varying (10)	lastname character varying (20)	total_orders bigint
1	4	Margaret	Peacock	156
2	3	Janet	Leverling	127
3	1	Nancy	Davolio	123
4	8	Laura	Callahan	104
5	2	Andrew	Fuller	96

-- INTERMEDIATE

-- 6. Retrieve the total sales per category by joining categories, products, and
-- order_details.
-- Show only categories with total sales above 50,000. Sort by total sales
-- descending.

-- get category attributes join categories->products->orderdetails
-- calculate sales , group by category attributes and filter by having
select c.categoryid , c.categoryname ,sum(d.unitprice*d.quantity*(1-d.discount)) as
total_sales
from categories c join products p on c.categoryid = p.categoryid
join order_details d on p.productid = d.productid
group by c.categoryid , c.categoryname
having sum(d.unitprice*d.quantity*(1-d.discount)) > 50000
order by total_sales desc;

	categoryid [PK] integer	categoryname character varying (15)	total_sales double precision
1	1	Beverages	267868.1797437187
2	4	Dairy Products	234507.2847417593
3	3	Confections	167357.22484282582
4	6	Meat/Poultry	163022.35939204693
5	8	Seafood	131261.73734469974

-- 7. List suppliers and the number of products they supply.
-- Show only suppliers who supply more than 5 products. Sort by product count
-- descending.

--select supplier attributes and product count using aggregation
-- filter using having and sort based on cnt
select s.supplierid,s.companyname,count(p.productid) as
total_products_supplied
from suppliers s join products p on s.supplierid = p.supplierid
group by s.supplierid,s.companyname
having count(p.productid) > 5
order by count(p.productid) desc;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

supplierid

[PK] integer

companyname

character varying (40)

total_products_supplied

bigint

-- 8. Get the average unit price per category.
 -- Show only categories where the average price is above 30. Sort by average
 -- price descending.

-- select category attributes , join categories->products
 -- group by the same and filter using having and order
 select c.categoryid,c.categoryname,round(avg(p.unitprice),2) as avg_price
 from categories c join products p
 on c.categoryid = p.categoryid
 group by c.categoryid,c.categoryname
 having round(avg(p.unitprice),2) > 30
 order by avg_price desc;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	categoryid [PK] integer	categoryname character varying (15)	avg_price numeric
1	6	Meat/Poultry	54.01
2	1	Beverages	37.98
3	7	Produce	32.37

-- 9. Display the total revenue generated per employee (orders + order_details).
 -- Show only employees generating more than 20,000 in revenue. Sort by
 -- revenue descending.

-- select emp attributes and then join the employees->orders->orderdetails
 -- grp by the same and then filter using having and order by
 select
 e.employeeid,e.firstname,e.lastname,sum(d.unitprice*d.quantity*(1-d.discount)) as
 total_sales
 from employees e join orders o on e.employeeid = o.employeeid
 join order_details d on o.orderid = d.orderid
 group by e.employeeid,e.firstname,e.lastname
 having sum(d.unitprice*d.quantity*(1-d.discount)) > 20000

Data Output Messages Notifications				
	employeeid [PK] integer	firstname character varying (10)	lastname character varying (20)	total_sales double precision
1	4	Margaret	Peacock	232890.84573612877
2	6	Michael	Suyama	73913.12943308528
3	2	Andrew	Fuller	166537.75483498262
4	7	Robert	King	124568.23481442405
5	9	Anne	Dodsworth	77308.06640252954

-- 10. Retrieve the number of orders shipped to each country.
 -- Show only countries with more than 10 orders. Sort by order count
 -- descending.

-- select shipcountry aggregate count and order by count
 -- after filtering using having
 select shipcountry , count(orderid) as num_of_orders
 from orders group by shipcountry
 having count(orderid)>10
 order by num_of_orders desc;

Data Output Messages Notifications		
	shipcountry character varying (15)	num_of_orders bigint
1	USA	122
2	Germany	122
3	Brazil	83
4	France	77
5	UK	56

-- 11. Find customers and the average order value (orders + order_details).
 -- Show only customers with average order value greater than 500. Sort by
 -- average descending.

-- select customer attributes and then join orers and orderdetails
 select c.customerid,c.contactname,avg(d.unitprice*d.quantity*(1-d.discount)) as
 avg_order_value
 from customers c join orders o on c.customerid = o.customerid
 join order_details d on o.orderid = d.orderid
 group by c.customerid,c.contactname
 having avg(d.unitprice*d.quantity*(1-d.discount)) > 500
 order by avg_order_value desc;

	customerid [PK] character (5)	contactname character varying (30)	avg_order_value double precision
1	QUICK	Horst Kloss	1282.294242831499
2	SIMOB	Jytte Petersen	1121.1398329909891
3	ERNSH	Roland Mendel	1028.1860622892261
4	HANAR	Mario Pontes	1026.2928118167445
5	PICCO	Georg Pippas	1005.6026067079085

-- 12. Get the top-selling products per category (by total quantity sold).
-- Show only products with total quantity sold above 200. Sort within category by
-- quantity descending.

-- get appropriate attributes group by category->then productid
-- do filtering and sorting
select c.categoryid,c.categoryname,p.productid,p.productname,sum(d.quantity)
as quantity_sold
from categories c join products p on c.categoryid = p.categoryid
join order_details d on p.productid = d.productid
group by c.categoryid,c.categoryname,p.productid,p.productname
having sum(d.quantity)>200
order by c.categoryname desc , quantity_sold desc;

	categoryid integer	categoryname character varying (15)	productid integer	productname character varying (40)	quantity_sold bigint
1	8	Seafood	40	Boston Crab Meat	1103
2	8	Seafood	41	Jack's New England Clam Cho...	981
3	8	Seafood	13	Konbu	891
4	8	Seafood	36	Inlagd Sill	805
5	8	Seafood	10	Ikura	742

-- 13. Retrieve the total discount given per product (order_details).
-- Show only products where total discount exceeds 1,000. Sort by discount
-- descending.

-- get product attributes and then join products->orderdetails
-- filter using having and order by calc discount
select p.productid,p.productname,sum(d.quantity*d.unitprice*d.discount) as
discount_amount
from products p join order_details d on p.productid = d.productid
group by p.productid,p.productname
having sum(d.quantity*d.unitprice*d.discount) > 1000
order by discount_amount desc;

	productid [PK] integer	productname character varying (40)	discount_amount double precision
1	38	Côte de Blaye	8587.465096551552
2	29	Thüringer Rostbratwurst	7367.728025485949
3	59	Raclette Courdavault	5140.300090570003
4	60	Camembert Pierrot	3460.520046980679
5	51	Manjimup Dried Apples	2922.9500460036097

-- 14. List employees and the number of unique customers they handled.
 -- Show only employees who handled more than 15 unique customers. Sort by
 -- count descending.

-- use distinct to include unique customers
 -- select emp attributes and join orders table and filter and sort
 select e.employeeid,e.firstname,e.lastname,count(distinct(o.customerid)) as
 unique_customers
 from employees e join orders o on e.employeeid = o.employeeid
 group by e.employeeid,e.firstname,e.lastname
 having count(distinct(o.customerid))>15
 order by unique_customers desc;

	employeeid [PK] integer	firstname character varying (10)	lastname character varying (20)	unique_customers bigint
1	4	Margaret	Peacock	75
2	1	Nancy	Davolio	65
3	3	Janet	Leverling	63
4	2	Andrew	Fuller	59
5	8	Laura	Callahan	56


```
-- 15. Find the monthly total sales (year + month) using orders and order_details.
-- Show only months where total sales exceed 30,000. Sort by year and month
-- ascending.
```

```
-- extract year and month and calculate total sales
-- grp by the same and sort by year and month
select extract(year from o.orderdate) as Year , extract(month from o.orderdate) as
Month,
sum(d.unitprice*d.quantity*(1-d.discount)) as total_sales
from orders o join order_details d on o.orderid = d.orderid
group by extract(year from o.orderdate) , extract(month from o.orderdate)
having sum(d.unitprice*d.quantity*(1-d.discount)) > 30000
order by year asc ,month asc;
```

Data Output Messages Notifications			
			
			
			
	year numeric 	month numeric 	total_sales double precision 
1	1996	10	37515.72491448447
2	1996	11	45600.04494312592
3	1996	12	45239.629965431996
4	1997	1	61258.0699304849
5	1997	2	38483.63494294286

PROCEDURES

-- 1.Create a stored procedure to insert a new customer. Use a transaction so that if any required value is missing, the insert is rolled back.

```
-- drop procedure add_customer;
create or replace procedure add_customer(
    n_customerid char(5),
    n_companyname varchar(40),
    n_contactname varchar(30),
    n_contacttitle varchar(30),
    n_address varchar(60),
    n_city varchar(15),
    n_region varchar(15),
    n_postalcode varchar(10),
    n_country varchar(15),
    n_phone varchar(24),
    n_fax varchar(24)
)
language plpgsql
as $$
begin
    if trim(coalesce(n_customerid, '')) = ''
       or trim(coalesce(n_companyname, '')) = ''
       or trim(coalesce(n_contactname, '')) = '' then

        raise notice 'required fields missing';
        rollback;
        return;
    end if;

    insert into customers (
        customerid,
        companyname,
        contactname,
        contacttitle,
        address,
        city,
        region,
        postalcode,
        country,
```

```

        phone,
        fax
    )
    values (
        n_customerid,
        n_companyname,
        n_contactname,
        n_contacttitle,
        n_address,
        n_city,
        n_region,
        n_postalcode,
        n_country,
        n_phone,
        n_fax
    );

    commit;
    raise notice 'customer added successfully';
end;
$$;

-- sample insert

call add_customer('AB123','abc corporation','laxman','manager','12 anna
salai','chennai',null,'600001',
    'india','9999999999',null);

```

Data Output	Messages	Notifications
-------------	----------	---------------

NOTICE: customer added successfully		
CALL		

Query returned successfully in 40 msec.		
---	--	--

-- 2. Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

```
-- select * from order_details;
```

```
create or replace procedure place_singleorder(
    n_customerid char(5),
    n_employeeid int,
    n_productid int,
    n_requireddate timestamp,
    n_shipvia int,
    n_freight numeric,
    n_shipname varchar(40),
    n_shipaddress varchar(60),
    n_shipcity varchar(15),
    n_shipregion varchar(15),
    n_shippostalcode varchar(10),
    n_shipcountry varchar(15),
    n_quantity smallint,
    n_discount real
)
language plpgsql
as $$
declare
    n_orderid int;
    n_unitprice numeric;
begin

    -- validate customer
    if not exists (
        select 1
        from customers
        where customerid = n_customerid
    ) then
        raise notice 'customer not found';
        rollback;
        return;
    end if;

    -- validate employee
    if not exists (
```

```
        select 1
        from employees
        where employeeid = n_employeeid
    ) then
        raise notice 'employee not found';
        rollback;
        return;
    end if;
```

```
-- validate shipper
if not exists (
    select 1
    from shippers
    where shipperid = n_shipvia
) then
    raise notice 'shipper not found';
    rollback;
    return;
end if;
```

```
-- fetch product price
select unitprice
into n_unitprice
from products
where productid = n_productid;
```

```
if not found then
    raise notice 'product not found';
    rollback;
    return;
end if;
```

```
insert into orders (
    customerid,
    employeeid,
    orderdate,
    requireddate,
    shipvia,
    freight,
    shipname,
    shipaddress,
    shipcity,
```

```

        shipregion,
        shippostalcode,
        shipcountry
    )
values (
    n_customerid,
    n_employeeid,
    current_timestamp,
    n_requireddate,
    n_shipvia,
    n_freight,
    n_shipname,
    n_shipaddress,
    n_shipcity,
    n_shipregion,
    n_shippostalcode,
    n_shipcountry
)
returning orderid into n_orderid;

insert into order_details (
    orderid,
    productid,
    unitprice,
    quantity,
    discount
)
values (
    n_orderid,
    n_productid,
    n_unitprice,
    n_quantity,
    n_discount
);

commit;
raise notice 'order placed successfully. order id: %', n_orderid;

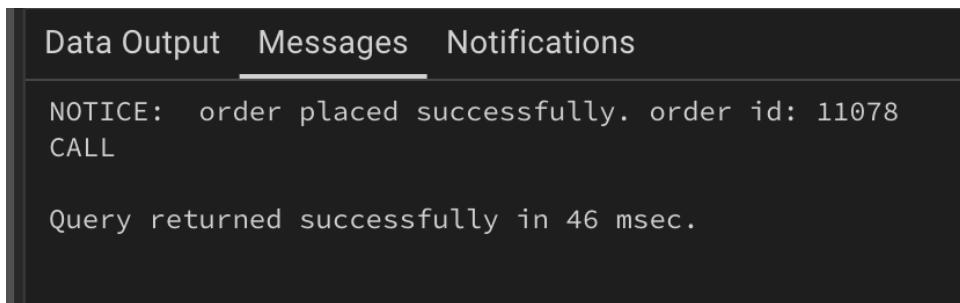
end;
$$;
select * from orders;
-- sample insert

```

```

call place_singleorder(
  'ALFKI'::char(5),
  4,
  11,
  '2026-05-01'::timestamp,
  3,
  32.38::numeric,
  'Vins et alcools Chevalier'::varchar,
  '59 rue de l'Abbaye'::varchar,
  'Reims'::varchar,
  null::varchar,
  '51100'::varchar,
  'France'::varchar,
  12::smallint,
  0::real
);

```



The screenshot shows a database client window with three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected and displays the following text:

```

NOTICE:  order placed successfully. order id: 11078
CALL

Query returned successfully in 46 msec.

```

-- 3.Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

```

create or replace procedure update_productstock(
  n_productid int,
  n_quantity smallint
)
language plpgsql
as $$
declare
  n_stock smallint;
begin

  -- fetch current stock
  select unitsinstock
  into n_stock
  from products

```

```

where productid = n_productid;

-- check if product exists
if not found then
    raise notice 'product not found';
    rollback;
    return;
end if;

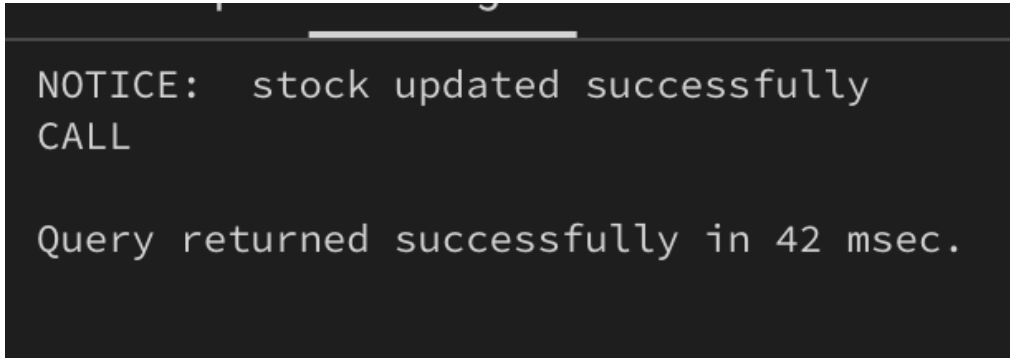
-- check stock availability
if n_stock < n_quantity then
    raise notice 'insufficient stock';
    rollback;
    return;
end if;

-- update stock
update products
set unitsinstock = unitsinstock - n_quantity
where productid = n_productid;

commit;
raise notice 'stock updated successfully';

end;
$$;
-- sample update
call update_productstock(11, 5::smallint);

```



```

NOTICE:  stock updated successfully
CALL

Query returned successfully in 42 msec.

```

-- 4. Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.


```

create or replace procedure cancel_order(
n_orderid int
)
language plpgsql
as $$
begin
    --check if order exists
    if not exists(
        select 1 from orders where orderid = n_orderid
    ) then
        raise notice 'Order doesnt exist';
        rollback;
        return;
    end if;

    --delete in orderdetails
    delete from order_details
    where orderid = n_orderid;

    --delete from orders
    delete from orders
    where orderid =n_orderid;
    commit;
    raise notice 'order deleted successfully or cancelled';

end

$$;
-- select * from orders;
call cancel_order(10248);

```

Data Output	Messages	Notifications
-------------	----------	---------------

NOTICE: order deleted successfully or cancelled		
CALL		

Query returned successfully in 50 msec.		
---	--	--

-- 5.Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

```
create or replace procedure transfer_supplierproducts(
    n_old_supplierid int,
    n_new_supplierid int
)
language plpgsql
as $$
begin

    -- check old supplier exists
    if not exists (
        select 1
        from suppliers
        where supplierid = n_old_supplierid
    ) then
        raise notice 'old supplier not found';
        rollback;
        return;
    end if;

    -- check new supplier exists
    if not exists (
        select 1
        from suppliers
        where supplierid = n_new_supplierid
    ) then
        raise notice 'new supplier not found';
        rollback;
        return;
    end if;

    -- transfer products
    update products
    set supplierid = n_new_supplierid
    where supplierid = n_old_supplierid;

    commit;
    raise notice 'products transferred successfully';

end;
```

```
$$;  
-- sample update  
-- select * from suppliers;  
call transfer_supplierproducts(1, 2);
```

Data Output Messages Notifications

```
NOTICE:  products transferred successfully  
CALL
```

```
Query returned successfully in 53 msec.
```

-- 6.Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

```
create or replace procedure update_categoryprices(  
    n_categoryid int,  
    n_percentage numeric  
)  
language plpgsql  
as $$  
begin  
  
    -- validate percentage  
    if n_percentage <= 0 then  
        raise notice 'percentage must be greater than zero';  
        rollback;  
        return;  
    end if;  
  
    -- validate category exists  
    if not exists (  
        select 1  
        from categories  
        where categoryid = n_categoryid  
    ) then  
        raise notice 'category not found';
```

```

        rollback;
        return;
    end if;

    -- update prices
    update products
    set unitprice=unitprice+(unitprice*n_percentage/100)
    where categoryid=n_categoryid;

    commit;
    raise notice 'category prices updated successfully';

end;
$$;
-- sample increase in price
-- select * from categories;
call update_categoryprices(1, 10);

```

Data Output	Messages	Notifications
NOTICE: category prices updated successfully CALL		
Query returned successfully in 56 msec.		

-- 7. Create a stored procedure to add a new product under an existing category and supplier. Rollback if the category or supplier does not exist.

```

create or replace procedure add_product(
    n_productname varchar(40),
    n_supplierid int,
    n_categoryid int,
    n_quantityperunit varchar(20),
    n_unitprice numeric,
    n_unitsinstock smallint,
    n_unitsonorder smallint,
    n_reorderlevel smallint,
    n_discontinued smallint
)

```

```
language plpgsql
as $$
begin

    -- validate supplier
    if not exists (
        select 1
        from suppliers
        where supplierid = n_supplierid
    ) then
        raise notice 'supplier not found';
        rollback;
        return;
    end if;

    -- validate category
    if not exists (
        select 1
        from categories
        where categoryid = n_categoryid
    ) then
        raise notice 'category not found';
        rollback;
        return;
    end if;

    -- insert product
    insert into products (
        productname,
        supplierid,
        categoryid,
        quantityperunit,
        unitprice,
        unitsinstock,
        unitsonorder,
        reorderlevel,
        discontinued
    )
    values (
        n_productname,
        n_supplierid,
        n_categoryid,
```

```

        n_quantityperunit,
        n_unitprice,
        n_unitsinstock,
        n_unitsonorder,
        n_reorderlevel,
        n_discontinued
    );

    commit;
    raise notice 'product added successfully';

end;
$$;
-- sample add
-- select * from products;
-- select * from categories;
call add_product('Maggie Noodles'::varchar,1,5,'10 boxes x 10
pkts'::varchar,25.50::numeric,100::smallint,20::smallint,10::smallint,0::smallint);

```

```

Data Output  Messages  Notifications
NOTICE:  product added successfully
CALL
Query returned successfully in 48 msec.

```

```

-- 8. Create a stored procedure to delete a customer only if the customer
has no orders. Use a transaction and rollback if orders exist.
create or replace procedure delete_customerifnoorders(
    n_customerid char(5)
)
language plpgsql
as $$
begin

    -- check customer exists
    if not exists (
        select 1
        from customers

```

```

        where customerid = n_customerid
    ) then
        raise notice 'customer not found';
        rollback;
        return;
    end if;

    -- check for existing orders
    if exists (
        select 1
        from orders
        where customerid = n_customerid
    ) then
        raise notice 'customer has orders and cannot be deleted';
        rollback;
        return;
    end if;

    -- delete customer
    delete from customers
    where customerid = n_customerid;

    commit;
    raise notice 'customer deleted successfully';

end;
$$;
-- sample delete
call delete_customerifnoorders('ALFKI');
```

Data Output Messages Notifications

```
NOTICE:  customer has orders and cannot be deleted
CALL
```

```
Query returned successfully in 42 msec.
```

-- 9. Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

```
create or replace procedure apply_orderdiscount(  
    n_orderid int,  
    n_discount real  
)  
language plpgsql  
as $$  
begin  
  
    -- check order exists  
    if not exists (  
        select 1  
        from orders  
        where orderid = n_orderid  
    ) then  
        raise notice 'order not found';  
        rollback;  
        return;  
    end if;  
  
    -- validate discount  
    if n_discount < 0 or n_discount > 1 then  
        raise notice 'discount must be between 0 and 1';  
        rollback;  
        return;  
    end if;  
  
    -- update discount for all order items  
    update order_details  
    set discount = n_discount  
    where orderid = n_orderid;  
  
    commit;  
    raise notice 'discount applied successfully';  
  
end;  
$;
```

```
-- sample update  
select * from orders;  
call apply_orderdiscount(10249, 0.15);
```


Data Output Messages Notifications

NOTICE: discount applied successfully
CALL

Query returned successfully in 84 msec.

-- 10. Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

```
create or replace procedure place_multiproductorder(
    n_customerid char(5),
    n_employeeid int,
    n_requireddate timestamp,
    n_shipvia int,
    n_freight numeric,
    n_shipname varchar(40),
    n_shipaddress varchar(60),
    n_shipcity varchar(15),
    n_shipregion varchar(15),
    n_shippostalcode varchar(10),
    n_shipcountry varchar(15),
    n_productids int[],
    n_quantities smallint[]
)
language plpgsql
as $$
declare
    n_orderid int;
    n_unitprice numeric;
    n_stock smallint;
    i int;
begin
    -- validate quantities and products
    if array_length(n_productids, 1) <> array_length(n_quantities, 1) then
        raise notice 'product and quantity array size mismatch';
```

```

        rollback;
        return;
    end if;

    -- insert order
    insert into orders (
        customerid,
        employeeid,
        orderdate,
        requireddate,
        shipvia,
        freight,
        shipname,
        shipaddress,
        shipcity,
        shipregion,
        shippostalcode,
        shipcountry
    )
    values (
        n_customerid,
        n_employeeid,
        current_timestamp,
        n_requireddate,
        n_shipvia,
        n_freight,
        n_shipname,
        n_shipaddress,
        n_shipcity,
        n_shipregion,
        n_shippostalcode,
        n_shipcountry
    )
    returning orderid into n_orderid;

    for i in 1..array_length(n_productids, 1) loop

        select unitprice, unitsinstock
        into n_unitprice, n_stock
        from products
        where productid = n_productids[i];
    end loop;

```

```

if not found then
    raise notice 'invalid product id: %', n_productids[i];
    rollback;
    return;
end if;

if n_stock < n_quantities[i] then
    raise notice 'insufficient stock for product id: %', n_productids[i];
    rollback;
    return;
end if;

insert into order_details (
    orderid,
    productid,
    unitprice,
    quantity,
    discount
)
values (
    n_orderid,
    n_productids[i],
    n_unitprice,
    n_quantities[i],
    0
);

update products
set unitsinstock = unitsinstock - n_quantities[i]
where productid = n_productids[i];

end loop;

commit;
raise notice 'multi product order placed successfully. order id: %',
n_orderid;

end;
$$;

```

Data Output Messages Notifications

NOTICE: multi product order placed successfully. order id: 11079
CALL

Query returned successfully in 39 msec.